TECHNICAL INTERVIEW PREPARATION — PART 2

AWS

Advanced Cloud & DevOps Engineer

50 New Expert-Level Questions | Q51–Q100 | Zero Repeats from Part 1

EC2 · ALB/NLB · RDS · S3 · Lambda · Route 53 · CloudFront · VPC · EKS
ECS · DynamoDB · Kinesis · Redshift · IAM · Terraform · DR · Compliance

Questions	Difficulty	New Services vs Part 1	Target Audience	Part
Q51–Q100 (50 questions)	Medium: 20 | Hard: 30	ECS, DynamoDB, Kinesis, Redshift, FIS, MSK, FinOps, DR, GDPR	Senior Cloud Engineer, Staff DevOps, Solutions Architect	Part 2 of 2

📖 How to Use This Guide

🎯 What they're testing — the underlying skill the interviewer is actually evaluating
✅ Key Answer Points — 6-8 bullets a senior/staff engineer would cover
💬 Follow-up Probe — the likely second question if you answer the first one well

This is Part 2 (Q51–Q100). Combine with Part 1 (Q1–Q50) for the complete 100-question set.

TABLE OF CONTENTS — PART 2 (Q51–Q100)

#	AWS Service / Topic	Questions	Q Count
1	EC2 — Advanced Scenarios (GPU Spot, ENI preserve, AZ failure)	Q51–Q53	3
2	ALB / NLB — Advanced (NLB vs ALB, rate limiting)	Q54–Q55	2
3	RDS — Advanced (Aurora Global, replica lag, encryption)	Q56–Q58	3
4	S3 — Advanced (Athena optimisation, data sovereignty)	Q59–Q60	2
5	Lambda — Advanced (idempotency, Step Functions, cold start)	Q61–Q63	3
6	Route 53 — Advanced (15-min RTO, split-horizon DNS)	Q64–Q65	2
7	CloudFront — Advanced (edge personalisation, cache hit ratio)	Q66–Q67	2
8	VPC — Advanced (PrivateLink, centralised egress, east-west)	Q68–Q70	3
9	EKS — Advanced (PCI, memory OOM, multi-tenancy)	Q71–Q73	3
10	IAM — Advanced (JIT access, policy evaluation, right-sizing)	Q74–Q76	3
11	Observability & CloudWatch — Advanced	Q77–Q78	2
12	ECS / Fargate (NLB vs ALB, CannotPullContainerError)	Q79–Q80	2
13	CI/CD — Advanced (CodeBuild optimisation, secret exposure)	Q81–Q82	2
14	Kinesis & Streaming (IoT ingestion, consumer lag)	Q83–Q84	2
15	DynamoDB (single-table design, hot partitions)	Q85–Q86	2
16	Redshift & Analytics (query tuning, real-time pipeline)	Q87–Q88	2
17	Networking Deep Dive (VPC packet flow, BGP Direct Connect)	Q89–Q90	2
18	FinOps — Advanced (programme design, RI strategy)	Q91–Q92	2
19	Disaster Recovery (4 DR strategies, DR testing automation)	Q93–Q94	2
20	Security — Advanced (GuardDuty IR, PII data protection)	Q95–Q96	2
21	Terraform — Advanced (prevent_destroy, state locking)	Q97–Q98	2
22	Governance & Compliance (GDPR, account migration)	Q99–Q100	2

🖥️ EC2 — Advanced Scenarios

Q51–Q53 · Part 2 Interview Questions

Q51 🖥️ EC2 — Advanced — Hard

You need to run a GPU-intensive machine learning training workload on EC2. The job takes 6 hours and can tolerate interruptions as long as it can resume. How do you architect this cost-effectively?

🎯 **What they're testing:** EC2 Spot for ML, checkpointing, p3/p4 instance types, fault tolerance

✅ **Key Answer Points:**

- Use p3.8xlarge or p4d.24xlarge Spot Instances — up to 90% cheaper than On-Demand for GPU
- Spot Interruption: 2-minute notice via EC2 metadata endpoint + EventBridge event
- Checkpointing: save model state to S3 every N steps — on resume, load latest checkpoint
- Use EC2 Auto Interruption Handler: Lambda catches interruption notice, writes checkpoint, terminates gracefully
- Spot Fleet with capacity-optimised strategy across multiple AZs and instance families
- FSx for Lustre: high-throughput scratch filesystem backed by S3 for training data
- AWS Batch: manages Spot lifecycle, retries, checkpointing for ML jobs automatically
- SageMaker Managed Spot Training: built-in checkpointing + Spot management — simplest option

💬 **Follow-up Probe:** How would you estimate the probability of Spot interruption for a specific instance type in a region, and which tool provides this data?

Q52 🖥️ EC2 — Advanced — Medium

An EC2 instance needs to be rebuilt from scratch but you need to preserve its data, IP address, and ENI attachment. Walk through the exact steps.

🎯 **What they're testing:** EC2 lifecycle, ENI detachment, EBS preservation, Elastic IP

✅ **Key Answer Points:**

- Elastic IP: disassociate from instance (preserves the IP) — reassociate to new instance after
- ENI: detach the secondary ENI — primary ENI cannot be detached while running
- EBS: snapshot all non-root volumes before termination
- For root volume: stop instance → detach root EBS → snapshot → launch new instance with new AMI → attach old data volumes
- Alternative: create AMI from the instance (includes root + all EBS volumes as snapshots)
- Launch new instance from AMI → attach Elastic IP → reattach ENI → reattach data volumes
- User Data: new instance picks up the same config from SSM Parameter Store or Secrets Manager
- Validate: run health checks before updating DNS / removing old instance from target group

💬 **Follow-up Probe:** What is the difference between an AMI backed by EBS and an AMI backed by instance store, and which is more appropriate for disaster recovery?

Q53 🖥️ EC2 — Advanced — Hard

Your EC2 instances are in an Auto Scaling Group across 3 AZs. One AZ goes down. Describe

exactly what happens and what you should proactively configure to handle this.

🎯 **What they're testing:** AZ failure handling, ASG rebalancing, capacity reserve, multi-AZ design

✅ **Key Answer Points:**

- ASG detects unhealthy instances in the failed AZ — replaces them in remaining AZs (rebalancing)
- Risk: if remaining AZs don't have capacity, scale-out may fail — request limit or Spot capacity exhausted
- Configure: Capacity Reservations in each AZ for critical baseline capacity
- Configure: On-Demand Capacity Reservations ensure instance availability regardless of AZ failure
- ASG Rebalancing: set to rebalance across AZs — after AZ recovers, instances redistributed
- ALB: already multi-AZ — continues routing to healthy AZs automatically
- Test AZ failure: use AWS Fault Injection Simulator (FIS) to simulate AZ outage in staging
- Monitor: AZRebalance activity in ASG history — verify it completes within SLA

💬 **Follow-up Probe:** How does AWS Fault Injection Simulator help you validate multi-AZ resilience, and what experiment template would you write to simulate a full AZ failure?

⚖️ ALB / NLB — Advanced

Q54–Q55 · Part 2 Interview Questions

Q54 ⚖️ ALB / NLB — Hard

When would you choose a Network Load Balancer over an Application Load Balancer? Give specific architectural scenarios where ALB would be the wrong choice.

🎯 **What they're testing:** NLB vs ALB trade-offs, Layer 4 vs Layer 7, use case selection

✅ **Key Answer Points:**

- NLB operates at Layer 4 (TCP/UDP/TLS) — ALB at Layer 7 (HTTP/HTTPS/gRPC)
- Choose NLB: ultra-low latency requirements (<1ms) — NLB has no HTTP processing overhead
- Choose NLB: non-HTTP protocols — MQTT, custom TCP, WebSockets at massive scale, gaming
- Choose NLB: static Elastic IPs needed — ALB IPs change; NLB IPs are static (important for whitelisting)
- Choose NLB: TLS passthrough — decrypt at application level for mTLS, not at LB
- Choose NLB: extreme throughput (millions of requests/sec) — NLB scales to millions, ALB to thousands
- Choose NLB: private link service provider — NLB required for AWS PrivateLink
- ALB wrong choice: database clients, SMTP, FTP, VoIP — these are not HTTP

💬 **Follow-up Probe:** A financial trading application requires microsecond latency and TLS termination must happen at the application for regulatory reasons. Which load balancer and why?

Q55 ⚖️ ALB / NLB — Medium

Your ALB access logs show a small number of client IPs generating 70% of your traffic. You need to rate-limit them without blocking legitimate burst traffic. How do you implement this?

🎯 **What they're testing:** WAF rate limiting, IP-based throttling, CloudFront, application-level controls

✅ **Key Answer Points:**

- AWS WAF rate-based rule: block IP if >X requests in 5-minute window (configurable)
- Rate-based rules can be scoped: ALL traffic, or specific URI path, or header condition
- Challenge action (CAPTCHA): challenge suspicious IPs instead of blocking — preserves legitimate users
- CloudFront in front of ALB: WAF at edge reduces load before reaching origin
- Graduated response: CAPTCHA first → count → block (avoid over-blocking)
- WAF IP Sets: maintain allow-list of known legitimate burst sources (CDNs, partners)
- API Gateway: if the traffic is API calls, use usage plans and API keys for per-client throttling
- Monitoring: WAF sampled requests in console, CloudWatch WAF metrics for blocked vs allowed counts

💬 **Follow-up Probe:** How would you implement per-user rate limiting (not just per-IP) for an API where users authenticate with JWT tokens, using Lambda@Edge?

🗄️ RDS — Advanced Scenarios

Q56–Q58 · Part 2 Interview Questions

Q56 🗄️ RDS — Advanced — Hard

You need to implement a global database that spans US, EU, and APAC regions with reads served locally and writes propagated within 1 second. Which AWS service and architecture would you use?

🎯 **What they're testing:** Aurora Global Database, multi-region architecture, replication lag, failover

✅ **Key Answer Points:**

- Aurora Global Database: one primary region + up to 5 secondary read-only regions
- Replication lag: <1 second (physical log-based replication, not SQL-level) — meets the requirement
- Each region: separate Aurora cluster serving local reads — latency < 10ms for regional users
- Writes: all go to primary region — application must route write traffic to primary endpoint
- Failover: promote a secondary region to primary in <1 minute (manual or automated)
- RPO ~1s, RTO <1 minute with managed failover — better than traditional replication
- Cost: cross-region data transfer + extra cluster cost per secondary region
- Alternative: DynamoDB Global Tables — multi-master, all regions accept writes, auto-conflict resolution

💬 **Follow-up Probe:** How would you handle write routing in an application using Aurora Global Database to ensure all writes go to the primary region, especially during a regional failover?

Q57 🗄️ RDS — Advanced — Medium

A developer is running long analytical queries on the production RDS instance that are degrading application performance. How do you fix this without a schema change?

🎯 **What they're testing:** RDS read replica routing, Performance Insights, query isolation

✅ **Key Answer Points:**

- Immediate: route analytical queries to a Read Replica — separate endpoint for OLAP traffic
- Performance Insights: identify top SQL by wait time, CPU, rows examined — find the slow queries
- Enable slow query log (MySQL) or pg_stat_statements (PostgreSQL) to track offenders
- RDS Proxy with read/write splitting: automatically routes SELECT to replica
- Resource groups (MySQL): cap CPU/memory for specific user connections
- Query timeout: set statement_timeout (PostgreSQL) to kill queries running >30 seconds
- Long-term: move analytics workload to Aurora Serverless v2 replica or export to Redshift/Athena
- CloudWatch metric: ReadLatency + DBLoad — set alarm to detect future incidents

💬 **Follow-up Probe:** How would you architect a near-real-time analytics pipeline from RDS to Redshift using AWS-native services without impacting RDS performance?

Q58 🗄️ RDS — Advanced — Hard

Describe a complete strategy for encrypting an existing unencrypted RDS production database with minimal downtime.

🎯 **What they're testing:** RDS encryption, KMS, snapshot encryption, migration with minimal downtime

✅ **Key Answer Points:**

- RDS encryption cannot be enabled in-place on an existing unencrypted instance
- Step 1: Take a snapshot of the unencrypted RDS instance
- Step 2: Copy the snapshot with encryption enabled (specify KMS CMK during copy)
- Step 3: Restore an encrypted RDS instance from the encrypted snapshot
- Step 4: Run DMS or application-level data validation between old and new instance
- Step 5: Update application connection strings — use multi-AZ for the new encrypted instance
- Step 6: DNS failover or blue-green cutover to the encrypted instance
- Downtime window: only the final cutover (minutes) + any replication catch-up
- KMS key: use Customer Managed Key (CMK) for key rotation control and cross-account access

💬 **Follow-up Probe:** What happens if the KMS CMK used to encrypt an RDS instance is accidentally deleted, and how do you prevent this scenario?

🪣 S3 — Advanced Scenarios

Q59–Q60 · Part 2 Interview Questions

Q59 🪣 S3 — Advanced — Hard

Your application uploads millions of small files (1-10KB) to S3 per day. Query performance on S3 with Athena is terrible. How do you optimise the architecture?

🎯 **What they're testing:** S3 data lake optimisation, file format, partitioning, compaction

✅ **Key Answer Points:**

- Problem: millions of small files = high Athena metadata overhead, many S3 API calls per query
- Solution 1: S3 Object Lambda or Glue ETL to compact small files into large Parquet files (128MB-1GB ideal)
- Solution 2: Kinesis Firehose with buffering (5 min / 128MB) — auto-compacts before writing to S3
- Partitioning: partition by year/month/day/hour — Athena prunes partitions and scans less data
- File format: convert CSV/JSON to Parquet or ORC — columnar, compressed, 10-100x faster queries
- Compression: Snappy for speed, ZSTD for best ratio — always compress Parquet files
- Glue Data Catalog: register partitions so Athena doesn't scan the whole bucket
- S3 Inventory: audit file size distribution to measure improvement after compaction

💬 **Follow-up Probe:** How does Athena cost model work, and how would you estimate query cost reduction after converting 1TB of CSV files to Parquet with Snappy compression?

Q60 🪣 S3 — Advanced — Medium

You need to implement strict data sovereignty — ensuring specific S3 buckets never store data outside a particular AWS region, even if misconfigured. How do you enforce this?

🎯 **What they're testing:** S3 data sovereignty, SCPs, bucket policies, regional enforcement

✅ **Key Answer Points:**

- AWS Organizations SCP: deny s3:CreateBucket unless condition StringEquals aws:RequestedRegion = eu-west-1
- S3 bucket policy: deny any PutObject from a principal outside the VPC endpoint or specific region
- S3 Object Ownership: enforce bucket owner controls — prevent other accounts from writing
- AWS Config Rule: s3-bucket-public-write-prohibited + custom rule checking bucket region
- IAM policy condition: aws:RequestedRegion ensures no cross-region API calls are made
- S3 Access Points with VPC restriction: data only accessible via a specific VPC endpoint
- CloudTrail + EventBridge: alert on any S3 replication rules pointing cross-region
- S3 Replication: explicitly block cross-region replication rules using SCP deny on s3:PutReplicationConfiguration

💬 **Follow-up Probe:** How would you audit all S3 buckets across 50 AWS accounts to identify any cross-region replication rules or public access settings using AWS Config Aggregator?

λ Lambda — Advanced Scenarios

Q61–Q63 · Part 2 Interview Questions

Q61 λ Lambda — Advanced — Hard

Your Lambda function processes SQS messages but is causing duplicate processing — the same message is processed more than once. Explain all the causes and implement a complete idempotency solution.

🎯 **What they're testing:** Lambda idempotency, SQS deduplication, at-least-once delivery, DynamoDB conditional writes

✅ **Key Answer Points:**

- Root cause: SQS Standard delivers at-least-once — same message can be delivered multiple times
- Cause 2: Lambda function fails mid-way, SQS retries after visibility timeout — partial processing repeated
- Cause 3: Lambda concurrency — two instances poll the same message within visibility timeout window
- Solution 1: SQS FIFO with ContentBasedDeduplication — deduplication at queue level (5-min window)
- Solution 2: Lambda Powertools Idempotency — DynamoDB-backed idempotency with configurable TTL
- Idempotency key: hash of the SQS MessageId or business key (orderId, eventId)
- DynamoDB conditional write: PutItem ConditionExpression attribute_not_exists(id) — fails if already processed
- Set SQS visibility timeout to 6x Lambda max execution time to prevent premature redelivery

💬 **Follow-up Probe:** What happens if your Lambda idempotency check succeeds (finds the key in DynamoDB) but the original processing actually failed mid-way — how do you detect and handle this partial failure state?

Q62 λ Lambda — Advanced Hard

You need to build a Lambda function that orchestrates 5 dependent microservice calls — some sequential, some parallel — with compensation logic if any step fails. How do you implement this?

🎯 **What they're testing:** Step Functions, saga pattern, distributed transactions, compensation

✅ **Key Answer Points:**

- Don't implement orchestration in Lambda code — use AWS Step Functions for stateful workflows
- Step Functions Express vs Standard: Express for high-volume short workflows; Standard for long-running with audit trail
- Parallel state: run independent service calls concurrently — reduces total latency dramatically
- Choice state: branch based on response from each service call
- Catch and Retry: each task has retry config (exponential backoff) + catch for compensation
- Saga pattern: each step has a compensating transaction — if step 4 fails, reverse steps 3, 2, 1
- Step Functions Activity: for long-running external tasks that poll for completion
- Cost: Express = $1/million state transitions; Standard = $0.025/1000 state transitions — pick based on volume

💬 **Follow-up Probe:** How would you implement the saga compensating transaction for a multi-step order flow (reserve inventory → charge card → confirm order) where the card charge succeeds but inventory reservation fails?

Q63 λ Lambda — Advanced — Medium

Your Lambda function's cold start latency is causing SLA breaches for a user-facing API. Provisioned Concurrency is too expensive. What are the alternatives?

🎯 **What they're testing:** Lambda cold start optimisation, SnapStart, runtime choice, init code

✅ **Key Answer Points:**

- SnapStart (Java only): Lambda takes a snapshot of initialised execution environment — cold start near zero
- Runtime choice: Python/Node.js cold starts are 100-300ms; Java/C# are 1-5s — switch runtime if possible
- Keep init code lean: only import what you need — lazy imports for infrequently used modules
- Connection pool outside handler: DB connections, SDK clients initialised once and reused
- Reduce deployment package size: smaller packages initialise faster — use Lambda Layers for shared deps
- Scheduled warm-up: EventBridge pings the function every 5 minutes to keep containers warm
- Lambda Extensions: use a lightweight extension instead of heavy SDK initialisation in function
- Architecture: for truly latency-sensitive paths, consider Fargate with pre-warmed containers instead

💬 **Follow-up Probe:** What is the exact sequence of events during a Lambda cold start, from the moment an invocation request arrives to when your handler code begins executing?

🌐 Route 53 — Advanced Scenarios

Q64–Q65 · Part 2 Interview Questions

Q64 🌐 Route 53 — Advanced — Hard

You need to implement a disaster recovery strategy with a 15-minute RTO across two AWS regions. How do you configure Route 53 to achieve automated failover within that window?

🎯 **What they're testing:** Route 53 failover, health checks, RTO/RPO, warm standby vs pilot light

✅ **Key Answer Points:**

- Route 53 Health Check: checks endpoint every 10s or 30s — failover in 3 consecutive failures = 30-90s detection
- After health check failure: Route 53 DNS TTL must expire for clients to get new record — set TTL to 60s
- Total failover time: health check detection (30-90s) + TTL (60s) + DNS propagation = ~3-5 minutes
- 15-min RTO achievable: warm standby (scaled-down replica running in DR region) flips traffic in minutes
- Pilot light: core services running, others need starting — RTO 10-30 min — may not hit 15-min target
- Multi-value answer routing: returns multiple healthy IPs — clients failover without DNS change
- Route 53 ARC (Application Recovery Controller): readiness gates + routing controls for deterministic failover
- Runbook: automate scaling up DR region before DNS switch using Lambda + EventBridge

💬 **Follow-up Probe:** What is the difference between Route 53 health check-based failover and Route 53 ARC routing controls, and why would a regulated financial service prefer ARC?

Q65 🌐 Route 53 — Advanced — Medium

Explain how Split-Horizon DNS works in AWS and design a scenario where you would use it for a hybrid cloud application.

🎯 **What they're testing:** Split-horizon DNS, Route 53 Resolver, private hosted zones, hybrid DNS

✅ **Key Answer Points:**

- Split-horizon: same domain resolves to different IPs depending on where the query originates
- AWS implementation: Route 53 Private Hosted Zone (PHZ) for internal + Public Hosted Zone for external
- Internal: myapp.example.com → 10.0.1.45 (private ALB). External: myapp.example.com → 54.x.x.x (public ALB)
- Scenario: internal traffic stays inside VPC (no NAT, lower latency); external traffic hits WAF + CloudFront
- PHZ association: associate the private hosted zone with specific VPCs — resolves differently per VPC
- Route 53 Resolver: for on-prem DNS → AWS PHZ resolution and AWS → on-prem DNS forwarding
- Resolver Inbound Endpoint: on-prem DNS servers forward .aws.internal queries here
- Resolver Outbound Endpoint: Lambda/EC2 forward .corporate.internal queries to on-prem DNS

💬 **Follow-up Probe:** How would you configure Route 53 Resolver rules to allow an EKS pod to resolve both AWS service endpoints and on-premise hostnames in a hybrid environment?

☁️ CloudFront — Advanced Scenarios

Q66–Q67 · Part 2 Interview Questions

Q66 ☁️ CloudFront — Advanced — Hard

You need to implement a globally distributed API with personalised content at the edge — user authentication, A/B testing, and dynamic content injection — all without hitting the origin for most requests. Design this.

🎯 **What they're testing:** Lambda@Edge, CloudFront Functions, edge personalisation, JWT auth, cache keys

✅ **Key Answer Points:**

- Viewer Request: CloudFront Function validates JWT token (lightweight, <1ms) — reject unauthorised at edge
- Origin Request: Lambda@Edge enriches request with user segment from DynamoDB edge replica or cookie
- A/B testing: Lambda@Edge reads experiment cookie → routes to different origins or injects variant header
- Dynamic content injection: Lambda@Edge modifies response body to inject personalised content
- Cache key customisation: include user-segment header in cache key — segment-level caching not user-level
- Edge KV store: CloudFront KeyValueStore — store experiment configs and feature flags at edge (no origin hit)
- Origin shield: reduce origin load with an additional caching layer between edge and origin
- Caution: Lambda@Edge has 1MB response size limit, 30s timeout, 128MB memory max

💬 **Follow-up Probe:** What are the restrictions on Lambda@Edge compared to regular Lambda, and how do these constraints influence the architecture of edge-based authentication systems?

Q67 ☁️ CloudFront — Advanced — Medium

Your CloudFront distribution is serving an e-commerce site. During a flash sale, origin requests spike 100x despite CloudFront. How do you investigate the cache miss rate and fix it?

🎯 **What they're testing:** CloudFront cache hit ratio, cache behaviour tuning, origin shield, TTL

✅ **Key Answer Points:**

- CloudWatch metric: CacheHitRate — if low, most requests are passing through to origin
- Common causes: too many unique query strings/cookies in cache key, low TTL, no-cache headers from origin
- Check Cache-Control headers from origin: no-cache or no-store forces every request to origin
- Fix: set minimum TTL in CloudFront cache policy to override origin's no-cache for static assets
- Cache key: remove unnecessary query strings and cookies from cache key — broaden cache hit surface
- Origin Shield: enable in the region closest to origin — additional caching layer reduces origin hits
- Compress: enable Gzip/Brotli compression — reduces response size and improves cache efficiency
- CloudFront real-time logs: stream to Kinesis Data Streams → analyse x-edge-result-type (Hit vs Miss vs Error)

💬 **Follow-up Probe:** How would you use CloudFront real-time logs and Kinesis Data Analytics to build a dashboard showing cache hit ratio per path in real time during a flash sale event?

🔒 VPC — Advanced Networking

Q68–Q70 · Part 2 Interview Questions

Q68 🔒 VPC — Advanced — Hard

You need to share a centralised service (internal API, Active Directory) with 30 VPCs across multiple accounts without using VPC peering or exposing the service to the internet. Design this.

🎯 **What they're testing:** AWS PrivateLink, VPC Endpoint Services, centralised shared services architecture

✅ **Key Answer Points:**

- AWS PrivateLink: expose service as a VPC Endpoint Service — consumers connect via Interface Endpoint
- Architecture: NLB in front of the service → create VPC Endpoint Service from NLB ARN
- Consumer VPCs: create Interface VPC Endpoint pointing to the provider's service name
- Traffic: stays entirely on AWS network — no internet, no VPC peering, no CIDR conflicts
- No overlapping CIDR concern: PrivateLink works regardless of CIDR overlap between VPCs
- Access control: Endpoint Service allows/denies specific principal ARNs (accounts, roles)
- DNS: private DNS name auto-resolves in consumer VPC to the endpoint's private IP
- Scaling: NLB handles cross-AZ routing to service targets — multi-AZ for HA
- Cost: per-endpoint-hour + per-GB — cheaper than running NAT Gateways across 30 VPCs

💬 **Follow-up Probe:** How does AWS PrivateLink differ from VPC Peering and Transit Gateway in terms of traffic direction, CIDR requirements, and use cases?

Q69 🔒 VPC — Advanced — Hard

Your security team requires all outbound internet traffic from EC2 instances to be inspected and filtered through a centralised proxy. How do you implement this at scale across a multi-VPC environment?

🎯 **What they're testing:** Centralised egress inspection, AWS Network Firewall, Transit Gateway, inspection VPC

✅ **Key Answer Points:**

- Centralised egress VPC: one VPC with NAT Gateway + AWS Network Firewall (or third-party appliance)
- Transit Gateway: all spoke VPCs route 0.0.0.0/0 traffic to egress VPC via TGW
- TGW Route Tables: separate route table per spoke — default route → egress VPC attachment
- Egress VPC routing: traffic → Network Firewall → NAT Gateway → Internet Gateway
- AWS Network Firewall: stateful inspection, domain-based filtering, IDS/IPS rules (Suricata-compatible)
- Domain allow-list: whitelist specific FQDNs (api.github.com, *.amazonaws.com) — block everything else
- VPC Flow Logs + Network Firewall logs → S3/CloudWatch Logs → Security Operations analysis
- Avoid: individual NAT Gateways per VPC — expensive and impossible to centralise inspection

💬 **Follow-up Probe:** How does AWS Gateway Load Balancer differ from AWS Network Firewall for inline traffic inspection, and when would you choose to deploy a third-party virtual appliance instead?

Q70 🔒 VPC — Advanced — Medium

A penetration test revealed that two microservices in the same VPC can communicate with each other directly. Security policy requires strict east-west traffic control. How do you implement this?

🎯 **What they're testing:** East-west traffic control, security groups, Network Firewall, micro-segmentation

✅ **Key Answer Points:**

- Security Groups: most effective east-west control — deny by default, allow only explicitly needed flows
- Service A SG: outbound only to Service B SG on port 8080 — no other egress allowed
- Service B SG: inbound only from Service A SG — not from any other SG
- Problem: SG rules are per-instance — as services scale, rules must be managed carefully
- AWS Network Firewall in inspection VPC with TGW: all east-west traffic routed through inspection
- Application-level mTLS: each service presents a certificate — unauthorised services cannot connect even if network allows
- AWS App Mesh with mTLS: service mesh enforces identity-based east-west policies + observability
- Zero-trust: combine SG micro-segmentation + mTLS + App Mesh — defence in depth

💬 **Follow-up Probe:** How would AWS App Mesh with Envoy sidecars complement Security Groups for east-west traffic control, and what additional capabilities does it provide beyond network-level controls?

☸ EKS — Advanced Scenarios

Q71–Q73 · Part 2 Interview Questions

Q71 ☸ EKS — Advanced — Hard

Your EKS workload needs to comply with PCI-DSS. What specific AWS and Kubernetes controls would you implement?

🎯 **What they're testing:** EKS security hardening, PCI compliance, pod security, network isolation

✅ **Key Answer Points:**

- EKS private endpoint: control plane API not accessible from internet — VPN/bastion only
- Node groups: use dedicated node groups for cardholder data environment (CDE) with taint/toleration isolation
- Pod Security Standards: enforce 'restricted' policy on CDE namespaces — no privileged containers
- Network Policies: block all traffic between CDE and non-CDE namespaces — allow only defined flows
- IRSA least privilege: each CDE pod has minimal IAM permissions — no wildcard actions
- Secrets: AWS Secrets Manager via External Secrets Operator — never store in etcd
- EKS audit logging: enable all log types — api, audit, authenticator, controllerManager, scheduler
- Container image scanning: ECR image scanning + Trivy in CI — block critical CVEs from deploying
- Runtime security: Falco or GuardDuty EKS Runtime Protection — detect anomalous container activity

💬 **Follow-up Probe:** PCI-DSS requires quarterly network scans and annual penetration tests. How would you scope an EKS cluster for a PCI assessment, and which components are in-scope vs out-of-scope?

Q72 ☸ EKS — Advanced — Medium

Your EKS pods are consuming more memory than expected, causing node pressure and OOMKilled events. How do you systematically diagnose and fix this?

🎯 **What they're testing:** EKS memory management, resource limits, VPA, OOMKilled debugging

✅ **Key Answer Points:**

- Check OOMKilled pods: kubectl get pods — 'OOMKilled' in STATUS or Last State
- kubectl describe pod: shows container exit code 137 (SIGKILL from OOM) and last memory usage
- Metrics Server + kubectl top pods: see current memory usage vs limits
- CloudWatch Container Insights: pod memory utilisation over time — find peak usage patterns
- Common causes: memory leak in application, undersized limits, JVM heap not bounded
- Fix: increase memory limit (request = limit for critical pods) — but first understand actual need
- Vertical Pod Autoscaler (VPA): in recommendation mode — analyses usage and suggests optimal request/limit
- For JVM: set -Xmx to 70% of container memory limit — JVM can exceed container limit causing OOM

💬 **Follow-up Probe:** How would you use VPA in recommendation mode alongside HPA without them conflicting, and what is the known limitation of running VPA and HPA on the same deployment?

Q73 ☸ EKS — Advanced — Hard

You need to run multi-tenant workloads on a single EKS cluster where different teams must be completely isolated — compute, network, RBAC, and billing. How do you implement this?

🎯 **What they're testing:** EKS multi-tenancy, namespace isolation, node isolation, RBAC, cost allocation

✅ **Key Answer Points:**

- Namespace per tenant: separate namespace per team with ResourceQuota and LimitRange
- RBAC: each team has a Role/ClusterRole bound to their namespace — no cross-namespace access
- Node isolation: dedicated node groups per tenant using taints + tolerations — noisy neighbour prevention
- Network isolation: NetworkPolicy deny-all default in each namespace + explicit allow rules
- Cost allocation: tag node groups per tenant + CloudWatch Container Insights namespace metrics → cost per team
- Secrets isolation: namespace-scoped Secrets — teams cannot read each other's secrets
- OPA/Kyverno: enforce naming conventions, label requirements, image registry restrictions per namespace
- Soft multi-tenancy (shared nodes) vs hard multi-tenancy (dedicated nodes): hard = better isolation, higher cost

💬 **Follow-up Probe:** What are the fundamental limitations of Kubernetes namespace-based multi-tenancy compared to separate clusters per tenant, and when would you recommend separate clusters despite the higher operational cost?

🔑 IAM & Security — Advanced

Q74–Q76 · Part 2 Interview Questions

Q74 🔑 IAM — Advanced — Hard

Design a just-in-time (JIT) privileged access system for production AWS access — engineers should not have standing privileged access but should be able to request it for a time-limited window.

🎯 **What they're testing:** JIT access, PAM, temporary credentials, break-glass, AWS IAM Identity Center

✅ **Key Answer Points:**

- IAM Identity Center (SSO): permission sets assigned temporarily — not standing access
- JIT workflow: engineer submits request (ServiceNow/Slack bot) → approval workflow → temporary permission set assigned
- Implementation: Lambda receives approval webhook → calls IAM Identity Center API to assign permission set with TTL
- Permission sets: scoped to specific account + read-only by default, escalation requires approval + MFA
- Session duration: IAM Identity Center session max 12h — set to 1-4h for privileged access
- CloudTrail: all actions during JIT session tagged with requester identity for audit
- Break-glass: emergency admin role with hardware MFA required — usage triggers immediate PagerDuty alert
- Review: weekly automated report of JIT access usage, approvals, and actions taken during sessions

💬 **Follow-up Probe:** How would you implement time-bounded AWS console access for a third-party vendor that needs to investigate a production issue, with full audit trail and automatic expiry?

Q75 🔑 IAM — Advanced — Medium

Explain the IAM policy evaluation logic when a request is made — walk through the exact order in which SCPs, permission boundaries, identity policies, and resource policies are evaluated.

🎯 **What they're testing:** IAM policy evaluation, SCPs, permission boundaries, resource policies, deny logic

✅ **Key Answer Points:**

- Step 1: Deny evaluation — explicit Deny in any policy immediately blocks the request
- Step 2: SCP check — if using AWS Organizations, SCP must allow the action (implicit deny if not listed)
- Step 3: Permission Boundary — if attached, identity policy cannot grant permissions outside the boundary
- Step 4: Identity policy — IAM user/role policy must Allow the action
- Step 5: Resource policy — if resource has a policy, it can allow cross-account access independently
- Cross-account: BOTH identity policy AND resource policy must allow the action
- Same account: either identity policy OR resource policy can allow (not both required)
- Session policies: AssumeRole with inline policy — further restricts the assumed role's permissions
- Effective permissions = intersection of: SCP ∩ PermissionBoundary ∩ IdentityPolicy

💬 **Follow-up Probe:** A developer has an IAM policy with Allow s3:* and their account SCP allows s3:GetObject only. They try to call s3:PutObject. What happens and why?

Q76 🔑 IAM — Advanced — Hard

Your security audit found that several Lambda functions have overly permissive IAM roles. How do you implement a systematic IAM right-sizing process across 200 Lambda functions?

🎯 **What they're testing:** IAM least privilege, Access Analyzer, CloudTrail-based policy generation, automation

✅ **Key Answer Points:**

- AWS IAM Access Analyzer: generates policy based on CloudTrail activity over a specified period
- Access Analyzer workflow: review CloudTrail → generate policy → review → apply as new permission boundary
- CloudTrail + Athena: query all IAM actions taken by each Lambda role over 90 days — find unused permissions
- Automation: Python script queries CloudTrail, identifies unused actions per role, generates minimal policy
- Phased approach: apply Permission Boundaries first (safe — doesn't break anything) then replace policies
- Testing: deploy updated policies to dev/staging first, run full test suite, monitor for AccessDenied errors
- CloudWatch metric filter: count AccessDenied events post-change — roll back if above threshold
- IAM Access Advisor: shows last accessed time per service per role — prune unused services

💬 **Follow-up Probe:** How would you use Permission Boundaries as a safety net during IAM right-sizing to ensure that even if you grant too much in the identity policy, the actual effective permissions are still bounded?

📊 Observability & CloudWatch — Advanced

Q77–Q78 · Part 2 Interview Questions

Q77 📊 Observability — Advanced — Hard

Design a complete observability stack for a microservices application on EKS — metrics, logs, traces, and alerts — using AWS-native services.

🎯 **What they're testing:** Observability pillars, CloudWatch Container Insights, X-Ray, OpenTelemetry

✅ **Key Answer Points:**

- Metrics: CloudWatch Container Insights — pod CPU, memory, network per namespace/deployment
- Custom metrics: Embedded Metric Format (EMF) from application code → CloudWatch custom metrics
- Logs: Fluent Bit DaemonSet on EKS nodes → CloudWatch Logs → Logs Insights for queries
- Traces: AWS Distro for OpenTelemetry (ADOT) collector on EKS → AWS X-Ray → ServiceLens
- X-Ray Service Map: visualise service dependencies, error rates, latency percentiles per service
- Synthetic monitoring: CloudWatch Synthetics canaries — test critical user journeys every minute
- Alerts: CloudWatch Composite Alarms — combine metrics for intelligent noise reduction
- Dashboards: CloudWatch Dashboard with service-level RED metrics (Rate, Errors, Duration) per service
- Grafana: Amazon Managed Grafana with CloudWatch + X-Ray data sources for unified observability UI

💬 **Follow-up Probe:** How would you implement SLO (Service Level Objective) tracking in CloudWatch — for example, 99.9% of requests must complete in under 200ms — with automated error budget burn rate alerts?

Q78 📊 Observability — Advanced — Medium

Your CloudWatch Logs bill is unexpectedly high. How do you identify the biggest contributors and implement a cost-reduction strategy without losing important log data?

🎯 **What they're testing:** CloudWatch Logs cost optimisation, log retention, subscription filters, S3 archiving

✅ **Key Answer Points:**

- CloudWatch Logs Insights: query to find top log groups by size — identify biggest contributors
- AWS Cost Explorer: filter by CloudWatch, break down by log ingestion vs storage vs queries
- Reduce retention: most log groups default to Never Expire — set appropriate retention (e.g., 30 days)
- Subscription filter → Kinesis Firehose → S3: archive logs to S3 for long-term at fraction of CW cost
- Log level control: set application log level to WARN/ERROR in production — remove DEBUG logs
- Sampling: for high-volume non-critical logs (health check access logs), sample 1% of requests
- VPC Flow Logs: most expensive source — reduce log format to custom (only needed fields) or use S3 destination
- Metric filter instead of keeping raw logs: extract key metrics from logs before they expire

💬 **Follow-up Probe:** How would you implement log aggregation where raw logs go to S3 after 7 days but structured events (errors, transactions) are retained in CloudWatch Logs for 90 days for fast querying?

🐳 ECS / Fargate

Q79–Q80 · Part 2 Interview Questions

Q79 🐳 ECS / Fargate — Hard

Compare ECS with EC2 launch type, ECS with Fargate, and EKS. When would you choose each for a containerised microservices platform?

🎯 **What they're testing:** Container orchestration comparison, operational overhead, cost, control

✅ **Key Answer Points:**

- ECS EC2: you manage EC2 nodes, more control over networking/storage, lower cost at scale, EC2 Spot support
- ECS Fargate: serverless containers — no node management, pay per vCPU/memory per second, higher per-unit cost
- EKS: Kubernetes — portable, large ecosystem, complex to operate, required for K8s-specific tooling
- Choose ECS EC2: cost-sensitive, high steady-state workloads, need GPU instances or specific instance types
- Choose Fargate: small teams without Ops expertise, variable/bursty workloads, fast start without cluster management
- Choose EKS: Kubernetes expertise in team, multi-cloud strategy, advanced scheduling needs, existing K8s tooling
- Fargate Spot: 70% cheaper, can be reclaimed — good for batch/dev workloads
- Hybrid: ECS Capacity Providers mix EC2 Auto Scaling + Fargate — On-Demand baseline, Fargate for bursts

💬 **Follow-up Probe:** For a startup with 3 DevOps engineers and 20 microservices expecting rapid growth, which would you recommend and why — ECS Fargate vs EKS with managed node groups?

Q80 🐳 ECS / Fargate — Medium

An ECS Fargate task is failing to start with 'CannotPullContainerError'. How do you troubleshoot and fix it?

🎯 **What they're testing:** ECS troubleshooting, ECR pull, task IAM role, VPC networking

✅ **Key Answer Points:**

- Check ECS task stopped reason: AWS console → task → Stopped Reason — exact error message
- CannotPullContainerError causes: no internet access, wrong image URI, ECR permissions denied
- ECR permissions: task execution role needs ecr:GetAuthorizationToken, ecr:BatchGetImage, ecr:GetDownloadUrlForLayer
- Network: Fargate task in private subnet needs NAT Gateway (or VPC Endpoints for ECR) to pull from ECR
- VPC Endpoints for ECR: com.amazonaws.<region>.ecr.api + com.amazonaws.<region>.ecr.dkr + S3 gateway endpoint
- Image URI: verify exact registry, repository, and tag in task definition
- ECR image exists: aws ecr describe-images --repository-name <repo> to confirm the tag exists
- Public image: Fargate cannot pull from Docker Hub without internet (NAT GW) — mirror to ECR instead

💬 **Follow-up Probe:** How would you implement a fully private ECS Fargate deployment — no internet access at all — using VPC Endpoints, and which endpoints are required?

🔧 CI/CD — Advanced

Q81–Q82 · Part 2 Interview Questions

Q81 🔧 CI/CD — Advanced — Hard

Your CodeBuild build times are 25 minutes — slowing down developer velocity. How do you systematically reduce this to under 5 minutes?

🎯 **What they're testing:** CI/CD optimisation, caching, parallelism, build performance

✅ **Key Answer Points:**

- CodeBuild cache: S3 cache for build dependencies (npm node_modules, Maven .m2, pip packages)
- Local cache: DOCKER_LAYER cache type — reuse Docker layer cache between builds
- Parallel builds: split test suite into shards — run in parallel CodeBuild projects
- Custom build image: pre-bake all dependencies into a custom Docker image in ECR — no download time
- Build spec optimisation: run lint/unit tests in parallel using '&' and 'wait' in build commands
- Artifact caching: if dependencies haven't changed (lock file unchanged), skip install phase
- CodeBuild fleet: reserved capacity — eliminates queue wait time for large teams
- GitHub Actions alternative: self-hosted runners on EC2 Spot — more control over build environment and caching

💬 **Follow-up Probe:** How would you implement a build cache invalidation strategy that automatically purges the cache when package.json or requirements.txt changes, but not on every commit?

Q82 🔧 CI/CD — Advanced — Medium

Your team accidentally pushed secrets (API keys) to a public GitHub repository. Walk through your incident response and long-term prevention strategy.

🎯 **What they're testing:** Secret exposure incident response, git history, credential rotation, prevention

✅ **Key Answer Points:**

- IMMEDIATE: Revoke the exposed credentials — IAM console, API provider dashboard — within minutes
- Assume compromise: treat the secret as fully exposed — check CloudTrail for usage since the commit
- GitHub: use GitHub Secret Scanning (auto-alerts on known secret patterns) — was it already detected?
- Remove from git history: git-filter-repo or BFG Repo Cleaner to purge the commit — but history is already public
- Force-push rewritten history: all team members must re-clone — coordinate before doing this
- New credentials: generate new keys/tokens after revoking old ones
- Prevention: git pre-commit hooks with detect-secrets or truffleHog — block secrets before commit
- Prevention: GitHub Advanced Security + CodeGuru Secrets Detection in CI pipeline
- Long-term: never use static credentials — use IAM roles, OIDC federation for CI/CD systems

💬 **Follow-up Probe:** How would you configure GitHub Actions to use AWS credentials without storing any long-lived access keys, using OIDC federation with IAM?

🌊 Kinesis & Streaming

Q83–Q84 · Part 2 Interview Questions

Q83 🌊 Kinesis & Streaming — Hard

You are ingesting 1 million events per second from IoT devices. Compare Kinesis Data Streams, Kinesis Firehose, MSK (Kafka), and SQS for this use case.

🎯 **What they're testing:** Streaming architecture, Kinesis vs Kafka, throughput design, IoT ingestion

✅ **Key Answer Points:**

- Kinesis Data Streams: 1MB/s per shard, 1000 records/s per shard — at 1M events/s need ~1000 shards
- KDS: replay within 7 days, multiple consumers, Lambda/KCL consumers — good for real-time processing
- Kinesis Firehose: delivery service to S3/Redshift/OpenSearch — no replay, no real-time processing
- MSK (Kafka): higher throughput per broker, more flexible partitioning, ecosystem (Kafka Connect, Streams)
- MSK: more operational complexity than KDS but better price/performance at massive scale
- SQS: not designed for streaming — no ordering guarantee, no replay, max 3000 msg/s per queue
- Recommendation: KDS for real-time processing + Firehose for archival to S3 in parallel
- IoT Core → Kinesis Data Streams: AWS IoT Rule Action natively integrates
- Cost: KDS $0.015/shard/hour + $0.014/million PUT payload units — at 1000 shards = $14.4/day just for shards

💬 **Follow-up Probe:** How would you handle a hot partition problem in Kinesis Data Streams where 80% of your IoT devices are sending data with the same partition key?

Q84 🌊 Kinesis & Streaming — Medium

Your Kinesis Data Streams consumer (Lambda) is falling behind — the IteratorAgeMilliseconds metric is growing. How do you scale out the consumer?

🎯 **What they're testing:** Kinesis consumer scaling, shard splitting, Lambda concurrency, KCL

✅ **Key Answer Points:**

- IteratorAgeMilliseconds: measures how far behind the consumer is — growing means consumer is too slow
- Solution 1: Increase Lambda concurrency limit — Lambda processes one shard per concurrent execution
- Each shard can only be read by one Lambda execution at a time — 1 shard = 1 concurrent Lambda
- If Lambda is the bottleneck: optimise Lambda code, increase memory (more CPU), reduce processing time
- If shards are the bottleneck: split shards — double the shards, doubles read throughput
- Shard split: each shard = 1MB/s read, 2MB/s write. Splitting doubles capacity but also cost
- Enhanced fan-out: each consumer gets dedicated 2MB/s throughput per shard — for multiple consumers
- KCL (Kinesis Client Library): multi-threaded consumer — better than Lambda for sustained high throughput

💬 **Follow-up Probe:** What is the trade-off between using Lambda with event source mapping vs a KCL application on EC2/ECS for Kinesis consumer, in terms of cost, latency, and operational complexity?

🔷 DynamoDB

Q85–Q86 · Part 2 Interview Questions

Q85 🔷 DynamoDB — Hard

Design a DynamoDB table schema for a multi-tenant SaaS application where each tenant has users, projects, and tasks, supporting efficient queries by tenant, by user, and by project status.

🎯 **What they're testing:** DynamoDB data modelling, single-table design, GSI, access patterns

✅ **Key Answer Points:**

- Single-table design: all entities in one table — partition key and sort key encode entity relationships
- PK: TENANT#<tenantId>, SK: USER#<userId> for users; PROJECT#<projectId> for projects; TASK#<taskId> for tasks
- Access pattern 1: get all users for tenant → Query PK=TENANT#123, SK begins_with USER#
- Access pattern 2: get all projects for tenant → Query PK=TENANT#123, SK begins_with PROJECT#
- Access pattern 3: get tasks by project → GSI1 PK=PROJECT#<id>, SK=STATUS#<status>
- GSI for status queries: GSI1PK=TENANT#<id>#STATUS#<status>, SK=CREATED_AT — supports paginated task lists
- Avoid: multiple tables per entity type — hot partition risk, harder transaction support
- DynamoDB Transactions: use for multi-item atomic writes (create task + update project count)

💬 **Follow-up Probe:** How would you implement optimistic locking in DynamoDB to prevent two users from simultaneously overwriting each other's changes to the same task?

Q86 🔷 DynamoDB — Medium

Your DynamoDB table is experiencing hot partition issues — one partition is handling 80% of all

traffic. How do you diagnose and fix this?

🎯 **What they're testing:** DynamoDB hot partitions, partition key design, write sharding, caching

✅ **Key Answer Points:**

- Diagnosis: CloudWatch DynamoDB metrics — ConsumedWriteCapacityUnits per second spikes; ThrottledRequests
- Root cause: poor partition key choice — e.g., using a status field (active/inactive) that 99% of items share
- Fix 1: Write sharding — append random suffix to partition key: userId#1, userId#2, ..., userId#10
- Application must query all shards and aggregate — trade-off between query complexity and throughput
- Fix 2: DAX (DynamoDB Accelerator): in-memory cache — absorbs read hot spots, reduces throttling
- Fix 3: SQS buffer for writes — queue burst writes, Lambda processes at a controlled rate
- Fix 4: Redesign partition key to use high-cardinality attribute (UUID, timestamp, userId)
- Adaptive capacity: DynamoDB auto-redistributes capacity to hot partitions — but has limits

💬 **Follow-up Probe:** How would you design a high-velocity leaderboard table in DynamoDB that handles 100,000 score updates per second without hot partition issues?

📈 Redshift & Analytics

Q87–Q88 · Part 2 Interview Questions

Q87 📈 Redshift & Analytics — Hard

Your Redshift cluster queries are taking 10x longer than expected. Walk through a systematic performance tuning process.

🎯 **What they're testing:** Redshift query optimisation, distribution keys, sort keys, vacuum, analyze

✅ **Key Answer Points:**

- Step 1: STL_QUERY + SVL_QLOG — identify slowest queries by elapsed_time
- Step 2: EXPLAIN plan — look for full table scans, massive data redistribution, nested loops
- Distribution key mismatch: joining two tables with different DISTKEY causes data reshuffling across nodes
- Fix DISTKEY: tables joined together should share the same distribution key (e.g., customer_id)
- Sort key: queries filtering on date should have date as SORTKEY — enables zone map pruning
- VACUUM: after large loads/deletes, VACUUM sorts rows and reclaims space — run regularly
- ANALYZE: updates statistics for query planner — without it, planner makes bad decisions
- Concurrency scaling: enable for burst query concurrency — auto-adds cluster capacity
- WLM (Workload Management): separate queues for short/long queries — prevent long queries from blocking

💬 **Follow-up Probe:** How does Redshift Spectrum differ from Redshift local storage in terms of pricing, performance, and when would you use it to query data that stays in S3?

Q88 📈 Redshift & Analytics — Medium

Design a near-real-time data pipeline from application databases (RDS PostgreSQL) to a Redshift data warehouse for business intelligence dashboards.

🎯 **What they're testing:** Data pipeline design, DMS CDC, Kinesis Firehose, Redshift ingestion patterns

✅ **Key Answer Points:**

- Source: RDS PostgreSQL with logical replication enabled (wal_level = logical)
- CDC: AWS DMS with ongoing replication task — captures INSERT/UPDATE/DELETE in near real-time
- DMS → Kinesis Data Streams: stream changes to Kinesis
- Kinesis Firehose: buffers and loads to S3 (intermediate landing zone) in Parquet format
- AWS Glue: transforms and enriches data — handles schema changes, type casting
- Redshift COPY command: load from S3 to Redshift — fastest bulk load method (parallel)
- Alternatively: Redshift Streaming Ingestion — Kinesis Data Streams → Redshift materialised view in seconds
- Refresh cadence: materialised view auto-refreshes; BI tool queries materialised view for sub-second queries

💬 **Follow-up Probe:** How would you handle schema evolution — for example, adding a column to the source RDS table — without breaking the downstream Redshift pipeline?

🌐 Networking Deep Dive

Q89–Q90 · Part 2 Interview Questions

Q89 🌐 Networking — Deep Dive — Hard

Explain how the AWS VPC networking stack works at a low level — from when a packet leaves an EC2 instance to when it arrives at another EC2 instance in a different AZ of the same VPC.

🎯 **What they're testing:** VPC packet flow, ENI, VPC router, inter-AZ networking, AWS hyperplane

✅ **Key Answer Points:**

- EC2 → ENI: packet leaves via the Elastic Network Interface (virtual NIC) attached to the instance
- ENI → VPC router: each subnet has an implicit router at the .1 address — packet hits the subnet router
- VPC router: checks the subnet route table — for intra-VPC traffic, routes directly to destination subnet
- Security Group evaluation: stateful — SG on source instance (outbound) and destination instance (inbound) checked
- Inter-AZ routing: packet traverses AWS backbone network — high-bandwidth, low-latency, encrypted
- Destination subnet router → destination ENI → destination EC2
- AWS Hyperplane: software-defined networking layer managing VPC routing at massive scale
- Data transfer cost: inter-AZ within same VPC = $0.01/GB each way — design to minimise cross-AZ traffic

💬 **Follow-up Probe:** How does VPC peering traffic flow differ from Transit Gateway traffic flow at the network level, and what are the performance and cost implications of each?

Q90 🌐 **Networking — Deep Dive** **Medium**

A network engineer asks you to explain AWS's BGP-based routing for Direct Connect. How does traffic engineering work with multiple Direct Connect connections?

🎯 **What they're testing:** Direct Connect, BGP, MED, AS PATH prepending, traffic engineering

✅ **Key Answer Points:**

- Direct Connect uses BGP (Border Gateway Protocol) to exchange routes between AWS and on-prem
- AS PATH prepending: make one path appear longer — traffic prefers the shorter AS PATH
- MED (Multi-Exit Discriminator): influence which AWS endpoint on-prem traffic prefers
- Active/Active: both connections advertise same routes with equal BGP attributes — traffic load-balanced
- Active/Passive: one connection is preferred (shorter AS PATH) — other is standby failover
- Direct Connect Gateway: terminate multiple VIF connections from one DX connection to multiple VPCs
- Site-to-Site VPN as backup: if DX BGP session drops, VPN route appears in AWS route table via BGP
- Route summarisation: advertise aggregate routes over DX to reduce BGP table size

💬 **Follow-up Probe:** How would you implement a Direct Connect failover to Site-to-Site VPN that is completely automatic and requires no human intervention when the BGP session drops?

💰 FinOps — Advanced

Q91–Q92 · Part 2 Interview Questions

Q91 💰 **FinOps — Advanced** **Hard**

Your organisation runs 500 AWS accounts. Design a FinOps programme from scratch — governance, visibility, optimisation, and accountability.

🎯 **What they're testing:** FinOps at scale, AWS Organizations, Cost and Usage Report, showback/chargeback

✅ **Key Answer Points:**

- Cost and Usage Report (CUR): single CUR for the Management Account covers all linked accounts
- Athena + QuickSight: query CUR for per-account, per-team, per-service cost breakdown
- Tagging strategy: mandatory tags (team, environment, project, cost-centre) enforced via SCP/Config
- Tag policy: AWS Organizations Tag Policy — enforces consistent tag values across accounts
- Showback: each team sees their own costs via QuickSight dashboard — no financial consequence yet
- Chargeback: allocate costs back to business units via internal billing — drives accountability
- AWS Budgets with Actions: automatically apply SCPs or stop resources when budget exceeded
- Savings Plans coverage report: identify accounts with low SP coverage — recommend commitments
- Monthly FinOps review: waste report (idle EC2, unattached EBS, unused RIs) presented to engineering leads

💬 **Follow-up Probe:** How would you handle the political challenge of implementing chargeback in an organisation where teams resist being charged for cloud costs they don't directly control (shared services, platform infrastructure)?

Q92 💰 **FinOps — Advanced** **Medium**

Your EC2 Reserved Instance coverage is at 30%. How do you increase it to 80% without over-committing and locking money into unused RIs?

🎯 **What they're testing:** Reserved Instance strategy, Savings Plans, coverage analysis, commitment management

✅ **Key Answer Points:**

- Analyse: Cost Explorer RI Coverage Report — which instance families/regions have low coverage
- Baseline vs variable: identify the steady-state minimum (never goes below N instances) — that's your RI target
- Convertible RIs: allow exchange to different instance type/region — less risk of stranding capacity
- Savings Plans over RIs: Compute Savings Plans cover all EC2 families, Fargate, Lambda — more flexible
- 1-year commitment: lower upfront cost than 3-year — less exposure to workload changes
- Staged approach: buy 3 months of On-Demand data then commit to 60% of baseline — review quarterly
- RI Marketplace: sell unused RIs — reduces risk of over-commitment
- AWS recommends: Coverage target 80% Savings Plans + 20% On-Demand buffer for flexibility

💬 **Follow-up Probe:** How do Compute Savings Plans differ from EC2 Instance Savings Plans in terms of flexibility and discount level, and which would you recommend for an organisation planning to migrate workloads from EC2 to Fargate over 18 months?

🔁 Disaster Recovery

Q93–Q94 · Part 2 Interview Questions

Q93 🔁 Disaster Recovery — Hard

Define the four AWS disaster recovery strategies — Backup & Restore, Pilot Light, Warm Standby, and Multi-Site Active-Active — and for each, give a realistic RTO/RPO and a specific AWS architecture.

🎯 **What they're testing:** DR strategy selection, RTO/RPO trade-offs, AWS DR architecture patterns

✅ **Key Answer Points:**

- Backup & Restore: RTO hours-days, RPO hours. AWS: S3 backups + CloudFormation to rebuild. Lowest cost.
- Pilot Light: RTO 10-30 min, RPO minutes. Core services running (DB replication active), app servers off. Start app servers on failover.
- Warm Standby: RTO 5-15 min, RPO seconds-minutes. Scaled-down replica running in DR region. Scale up on failover.
- Multi-Site Active-Active: RTO near-zero, RPO near-zero. Both regions serving traffic. Route 53 latency routing.
- Backup & Restore tools: AWS Backup, cross-region snapshot copies, S3 Cross-Region Replication
- Pilot Light AWS: RDS cross-region read replica (promote on failover), AMIs copied to DR region
- Warm Standby AWS: ASG at min capacity in DR, RDS promoted, Route 53 health check triggers failover
- Cost: B&R cheapest → Pilot Light → Warm Standby → Active-Active most expensive

💬 **Follow-up Probe:** How does the AWS Well-Architected Framework's Reliability pillar guide the selection of DR strategy, and what formula would you use to calculate the cost justification for upgrading from Pilot Light to Warm Standby?

Q94 🔁 Disaster Recovery — Medium

How would you implement automated disaster recovery testing — verifying that your DR environment actually works without waiting for a real disaster?

🎯 **What they're testing:** DR testing automation, AWS Fault Injection Simulator, runbook automation, game days

✅ **Key Answer Points:**

- Schedule: quarterly DR tests — unannounced to the team to test real readiness
- AWS FIS (Fault Injection Simulator): inject EC2 terminations, AZ outages, network latency — controlled chaos
- Automated DR drill: EventBridge scheduled rule → Step Functions workflow → execute DR runbook
- DR runbook in Systems Manager Automation: codify every failover step — automated, auditable
- Test: failover to DR → run smoke tests → measure actual RTO → failback to primary
- Metrics to capture: time from incident detection to first request served from DR = actual RTO
- Compare to target: if actual RTO > target RTO → find bottleneck → improve → re-test
- Game days: full team exercises with simulated production incidents — builds muscle memory

💬 **Follow-up Probe:** How would you use AWS Resilience Hub to continuously validate that your application meets its defined RTO and RPO targets as the architecture evolves?

🔐 Security — Advanced

Q95–Q96 · Part 2 Interview Questions

Q95 🔐 Security — Advanced Hard

GuardDuty has raised a finding: 'UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS'. Walk through your incident response.

🎯 **What they're testing:** GuardDuty incident response, credential compromise, forensics, containment

✅ **Key Answer Points:**

- Finding means: EC2 instance credentials (from metadata) are being used from outside AWS
- IMMEDIATE (containment): attach deny-all SCP or revoke session by adding deny policy to the IAM role
- Isolate the instance: detach from ASG (prevent replacement), change SG to block all traffic
- Do NOT terminate the instance immediately — preserve forensic evidence first
- Forensic snapshot: create EBS snapshot of all volumes for offline analysis
- Memory dump: if possible, use SSM Run Command to dump process list, network connections, memory
- CloudTrail analysis: what API calls were made using the exfiltrated credentials? What was accessed?
- Root cause: how did attacker get credentials? SSRF vulnerability, malicious process on the instance?
- IMDS: switch to IMDSv2 on all instances (prevents SSRF-based credential theft)
- Post-incident: patch vulnerability, rotate all credentials, review GuardDuty findings across fleet

💬 **Follow-up Probe:** How does IMDSv2 (Instance Metadata Service v2) prevent SSRF-based credential exfiltration, and what is the mechanism difference from IMDSv1?

Q96 🔐 Security — Advanced — Hard

Design a comprehensive data protection strategy for PII data flowing through an AWS data pipeline — from ingestion to storage to consumption.

🎯 **What they're testing:** Data protection, encryption, tokenisation, DLP, data classification

✅ **Key Answer Points:**

- Classification: tag all PII fields at ingestion — AWS Macie auto-discovers PII in S3
- Encryption in transit: TLS 1.2+ enforced for all API endpoints, ALB, and inter-service communication
- Encryption at rest: KMS CMK per data classification level — PII uses dedicated CMK with strict key policy
- Tokenisation: replace PII (SSN, card numbers) with non-reversible tokens at ingestion — only token vault maps back
- Data masking: Lambda transformation before loading to analytics (last 4 digits of phone only)
- Access control: IAM policy conditions — kms:CallerAccount, kms:ViaService — restrict who can decrypt
- Amazon Macie: continuously scan S3 for PII exposure — alert on publicly accessible PII buckets
- VPC Endpoints: data never leaves AWS network during processing
- Audit trail: CloudTrail data events on S3 — every GetObject on PII bucket logged
- Data retention: S3 Object Lock for compliance hold; lifecycle policy for automatic deletion

💬 **Follow-up Probe:** How would you implement field-level encryption for a DynamoDB table storing medical records, where only specific Lambda functions with specific KMS grants can decrypt certain attributes?

🔧 Terraform — Advanced

Q97–Q98 · Part 2 Interview Questions

Q97 🔧 Terraform — Advanced — Hard

You need to destroy a Terraform-managed resource that has prevent_destroy = true and is in a shared module used by 5 teams. How do you approach this safely?

🎯 **What they're testing:** Terraform state management, lifecycle rules, safe destroy process, coordination

✅ **Key Answer Points:**

- Never remove prevent_destroy without team coordination and approval process
- Step 1: open an RFC/change request — document why the resource must be destroyed, impact on 5 teams
- Step 2: remove prevent_destroy = true from the module — commit to a feature branch, review
- Step 3: run terraform plan — verify only the intended resource is flagged for destruction
- Step 4: schedule change window — notify all 5 teams consuming the module
- Step 5: terraform destroy -target=<specific-resource> — never destroy without -target for shared modules
- Step 6: update all 5 team module references to remove the dependency before running destroy
- Alternative: import a replacement resource into state before destroying the old one (zero downtime)
- State manipulation: terraform state rm as last resort — removes resource from state without destroying it

💬 **Follow-up Probe:** When would you use terraform state rm vs terraform destroy vs terraform import, and what are the risks of each operation on a shared production state file?

Q98 🔧 Terraform — Advanced — Medium

Your Terraform apply in CI/CD is failing because two pipeline runs are competing for the same state lock. How do you design a pipeline that prevents this without serialising all deployments?

🎯 **What they're testing:** Terraform state locking, DynamoDB lock, parallel pipelines, workspace strategy

✅ **Key Answer Points:**

- Terraform remote state with DynamoDB locking: only one apply can run per state file at a time
- Problem: monorepo with 20 services — every PR triggers apply on the same state file
- Solution: separate state file per service — each service has its own S3 key and DynamoDB lock
- Directory structure: each service in environments/<env>/<service>/ with its own backend config
- Pipeline: detect which service changed (git diff) → only run apply for that service's state
- Queue-based apply: Atlantis or Terraform Cloud manage a per-state queue — no collision
- Blast radius reduction: separate state files also limit the damage of a failed apply
- Terraform workspaces: only useful for lightweight environment variation — not a replacement for state isolation

💬 **Follow-up Probe:** How would you implement a drift detection system that runs terraform plan on all 20 services every night and sends a Slack alert if any infrastructure has been manually changed outside of Terraform?

📋 Governance & Compliance

Q99–Q100 · Part 2 Interview Questions

Q99 📋 Governance & Compliance — Hard

Your organisation is expanding to the EU and must comply with GDPR. What AWS architecture changes and controls do you implement?

🎯 **What they're testing:** GDPR compliance, data residency, AWS config rules, data subject rights

✅ **Key Answer Points:**

- Data residency: all EU customer data must stay in EU Regions — use SCPs to deny non-EU region usage for EU accounts
- Data classification: AWS Macie to identify PII across S3 — tag and restrict access to PII data stores
- Encryption: all PII encrypted with CMK — key usage logged in CloudTrail
- Data subject rights: implement API to handle right-to-erasure (delete user data across RDS, DynamoDB, S3)
- DynamoDB TTL: auto-delete records after retention period
- S3 Object Expiry: lifecycle policy to delete after retention period
- Data Processing Agreements: AWS signs DPAs — use AWS services that are GDPR-compliant by default
- Breach notification: CloudWatch Alarms + GuardDuty for anomalous data access → 72-hour GDPR notification clock
- AWS Artifact: download AWS compliance reports (SOC 2, ISO 27001) for your compliance evidence pack

💬 **Follow-up Probe:** How would you implement a technical solution to handle a data subject access request (DSAR) — where a user requests all data held about them — when their data is spread across RDS, DynamoDB, S3, and CloudWatch Logs?

Q100 📋 Governance & Compliance — Hard

You are the cloud architect for a company being acquired. The acquirer wants all workloads migrated to their AWS Organisation within 90 days. Design the migration strategy.

🎯 **What they're testing:** AWS account migration, Organizations, resource migration, minimal downtime

✅ **Key Answer Points:**

- Phase 1 (Days 1-14): inventory all resources using AWS Config + CloudFormation stack exports across all accounts
- Phase 2 (Days 15-30): establish AWS account vending in target Org, set up Control Tower, SSO, networking
- Phase 3 (Days 31-60): migrate workloads using AWS Migration Hub — track progress centrally
- Account migration: move AWS accounts between Organizations using Organizations invite (no resource migration needed for account-level)
- Resource migration: use AWS Application Migration Service (MGN) for EC2 workloads — continuous replication + cutover
- Database: DMS for RDS migration; Aurora snapshot cross-account restore
- S3: S3 Replication cross-account for data migration; update bucket policies to allow target account
- DNS: Route 53 hosted zone association — transfer zones to target account's hosted zones
- IAM: all IAM roles/users must be recreated — cannot move IAM resources between accounts

► Phase 4 (Days 61-90): cutover production workloads in maintenance windows; decommission source accounts

💬 **Follow-up Probe:** What are the services and resources that CANNOT be migrated between AWS accounts (only recreated), and how does this affect your migration timeline and approach?

COMBINED TOPICS REFERENCE — PART 1 + PART 2

Topic Area	Part 1 Questions	Part 2 Questions	Total
EC2	Q1–Q5 (5)	Q51–Q53 (3)	8
ALB / NLB	Q6–Q10 (5)	Q54–Q55 (2)	7
Auto Scaling	Q11–Q14 (4)	—	4
RDS	Q15–Q19 (5)	Q56–Q58 (3)	8
S3	Q20–Q23 (4)	Q59–Q60 (2)	6
Lambda	Q24–Q27 (4)	Q61–Q63 (3)	7
Route 53	Q28–Q30 (3)	Q64–Q65 (2)	5
CloudFront	Q31–Q32 (2)	Q66–Q67 (2)	4
VPC & Networking	Q33–Q35 (3)	Q68–Q70, Q89–Q90 (5)	8
Security Groups & NACLs	Q36–Q37 (2)	—	2
EKS	Q38–Q40 (3)	Q71–Q73 (3)	6
IAM & Security	Q41–Q42, Q95–Q96 (4)	Q74–Q76 (3)	7
CloudWatch & Observability	Q43–Q44 (2)	Q77–Q78 (2)	4
Cost / FinOps	Q45–Q46 (2)	Q91–Q92 (2)	4
CI/CD & IaC	Q47–Q48 (2)	Q81–Q82, Q97–Q98 (4)	6
SQS / SNS / EventBridge	Q49–Q50 (2)	—	2
ECS / Fargate	—	Q79–Q80 (2)	2
DynamoDB	—	Q85–Q86 (2)	2
Kinesis & Streaming	—	Q83–Q84 (2)	2
Redshift & Analytics	—	Q87–Q88 (2)	2
Disaster Recovery	—	Q93–Q94 (2)	2
Governance & Compliance	—	Q99–Q100 (2)	2